

Технологии программирования

Вводная лекция

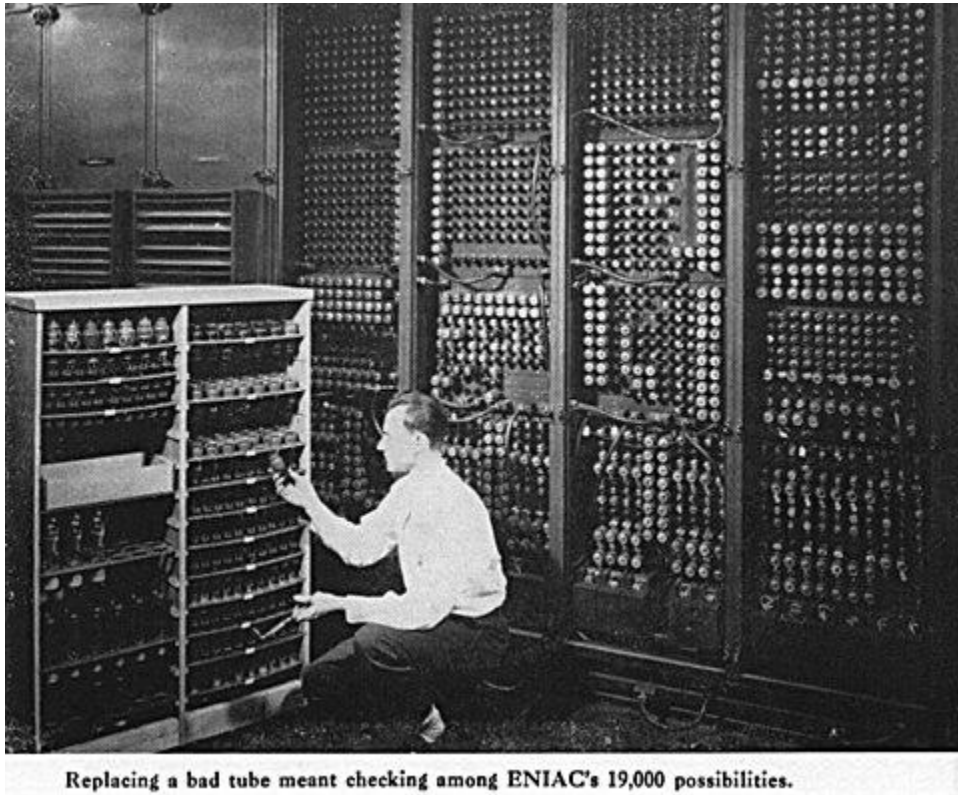
О чём курс?

- Устройство виртуальной машины Java
- Типы данных
- Объектно-ориентированное программирование на Java
- Обработка ошибок
- Generics (Обобщения)
- Java коллекции
- Функциональное программирование на Java
- Работа с базами данных
- Средства сборки, тестирование
- Spring
- Микросервисы

Предпосылки появления Java

Зачем создаются новые или совершенствуются существующие языки программирования?

Происхождение



Перфокарты

n.b. Супер компьютер тех лет
мог съедать
125 карт/минуту на ввод,
и отдавать
100 карт/минуту на вывод

40x

Происхождение



FORTRAN, BASIC, COBOL, Pascal, Ассемблер и т.д.

Зарождение основных парадигм программирования, множество новых идей и спорных аспектов в различных концепциях

60x

Происхождение

Разбиение на множество разных концепций в зависимости от типов задач которые нужно решать (ПП, СП, ООП, ФП и т.д.)

Основной недостаток: ни один из представленных в те времена ЯП не мог удовлетворить одновременно всем следующим критериям:

- простота использования;
- предоставляемые возможности;
- безопасность;
- эффективность;
- устойчивость;
- расширяемость.

Происхождение

Си – современный этап развития языков программирования. В целом, удовлетворял всем вышесказанным критериям и обладал очень важной особенностью: был создан программистами для программистов. Предыдущие ЯП создавались в академических целях или бюрократическими организациями.

70х

Происхождение

- В 80-е годы наступил период, который можно условно назвать временем консолидации. Язык С++ объединил в себе черты объектно-ориентированного и системного программирования
- размер и сложность программ выросли настолько, что понимать код, написанный в структурной парадигме на структурном ЯП стало очень сложно. Здесь на сцену вышла объектно-ориентированная парадигма, которая решала вопрос размера программ, разбиением ее на классы, модули и т.д. А первым языком, получившим большое признание и широко используемым в разработке ПО, стал С++

Происхождение

К 90-ым годам уже происходила массовая домашняя компьютеризация и разработка программного обеспечения перестала быть чем то сугубо промышленным и начала становится как хобби.

На рынке появлялось всё больше производителей компьютерных систем и стали появляться новые требования к ЯП, а именно возможности запуска на одного и того же кода на различных платформах.

Java

- Исходя из этой «хотелки» начали работу на Java, который смог бы запускаться на любом холодильнике, микроволновке, мобильном телефоне и т.д.
- А еще в этот же момент начала активно расширяться всемирная паутина (веб). И благодаря этому факту вопрос о программах, которые смогли бы работать под любым типом процессора и любой ОС стала еще острее.
- Исходя из всех вводных, был создан ЯП, который позволял запускать единожды скомпилированную программу на любой системной архитектуре и соответствовал всем высоким стандартам C++ (хотя на самом деле в некоторых аспектах уступал ему).

Managed / Unmanaged Code

Высокоуровневые языки программирования обычно не создают объектный модуль или машинный код, а используют только промежуточный язык. Код, созданный на этом промежуточном языке, затем подается компилятору данного языка, который создает финальный объектный модуль или машинный код. Обычно это делается для облегчения процесса оптимизации или увеличения портируемости.

Байт-код и JVM

Байт-код Java — набор инструкций, исполняемых виртуальной машиной Java. Каждый код операции байт-кода — один байт; используются не все 256 возможных значений кодов операций, 51 из них зарезервирован для использования в будущем.

Для программирования на языке Java или других JVM-совместимых языках знание особенностей байт-кода не обязательно, тем не менее, «понимание байт-кода и понимание механизмов его генерации компилятором Java помогает Java-программисту так же, как и знание языка ассемблера помогает программисту, пишущему на Си или C++»

<https://habr.com/ru/post/568402/>

Java

- JVM – Java virtual machine, виртуальная машина, способная запускать и транслировать байт-код в машинные инструкции.
- JRE – Java runtime environment, среда выполнения для Java, содержит библиотеки классов, загрузчик классов, JVM.
- JDK – Java development kit, средства, позволяющие разрабатывать на Java. Основным на текущий момент является OpenJDK

JVM

- Если говорить, очень поверхностно и глобально, то она состоит из трех частей:
- спецификация – набор правил, диктующий, как должна быть реализована JVM. Веселит то, что сводится она примерно к: «JVM должна правильно запускать программы, написанные на Java»;
- реализация – реальная программа, которая будет запускать и позволять разрабатывать программы, написанные на Java;
- экземпляр – по факту, оболочка над вашим кодом, которая его исполняет и заботится о том, как она это делает. Например, Minecraft – это программа, которая написана на Java, но запускается она в экземпляре JVM со всеми вытекающими, (отдельно выделенными ресурсами железа и своими процессами по управлению этими ресурсами).

Code example

```
package ru.butenko.springdatatest;

import java.time.LocalDate;

public class Dog {

    public String name;
    private LocalDate birthdate;

    public int calculateAge() {
        return LocalDate.now().getYear() - this.birthdate.getYear();
    }
}
```

Byte-Code Example

```
// class version 62.0 (62)
// access flags 0x21
public class ru/butenko/springdatatest/Dog {

    // compiled from: Dog.java

    // access flags 0x1
    public Ljava/lang/String; name

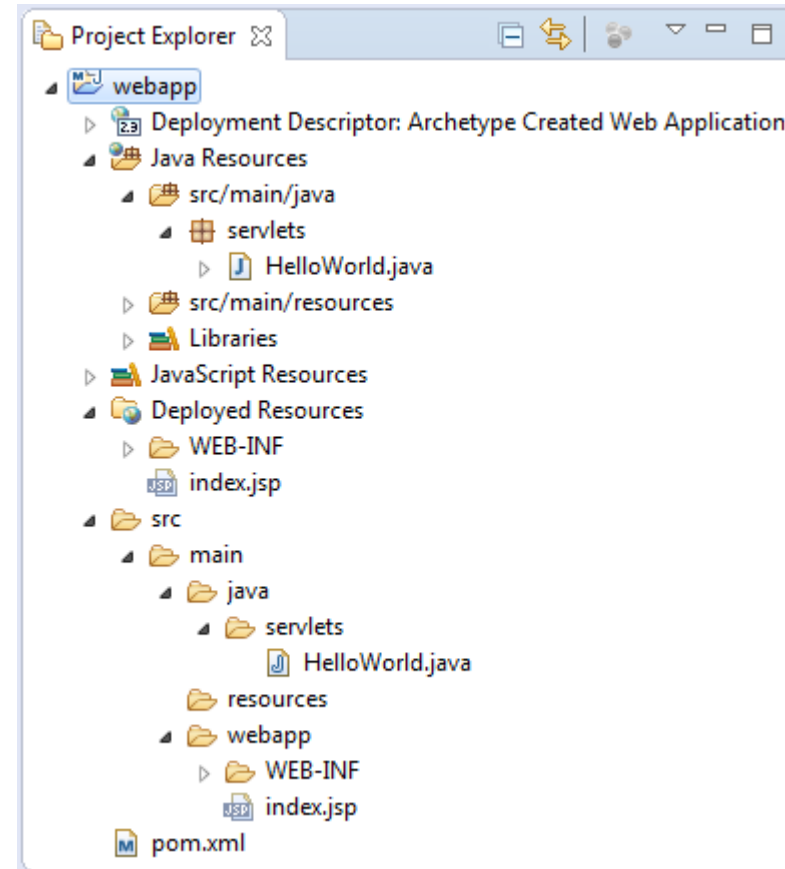
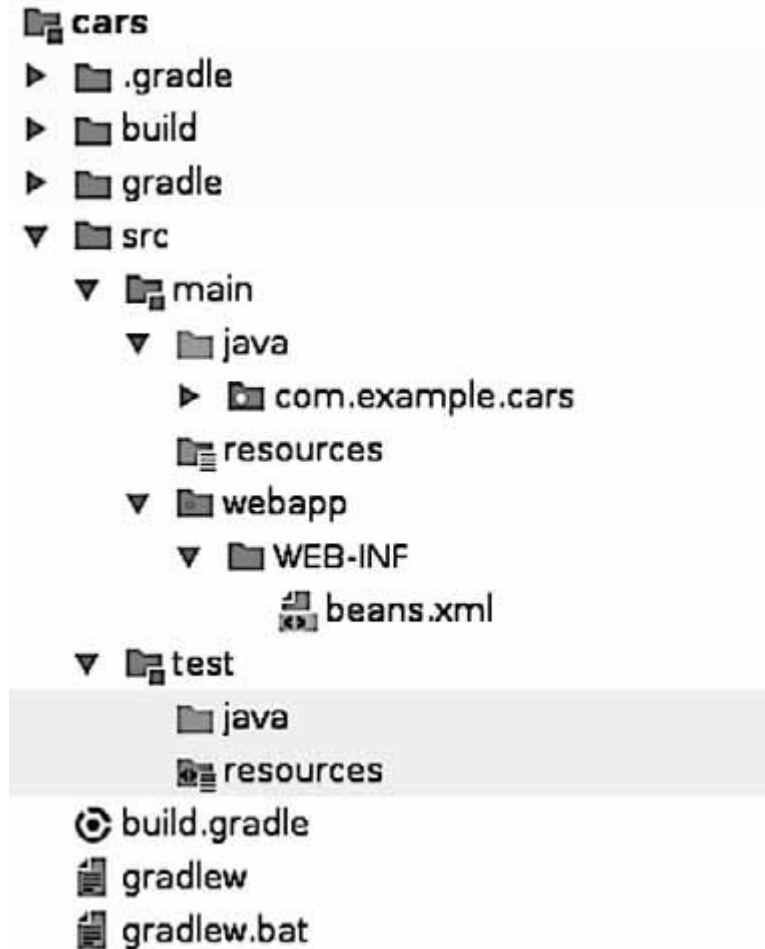
    // access flags 0x2
    private Ljava/time/LocalDate; birthdate

    // access flags 0x1
    public calculateAge()I
    L0
    LINENUMBER 11 L0
    INVOKESTATIC java/time/LocalDate.now ()Ljava/time/LocalDate;
    INVOKEVIRTUAL java/time/LocalDate.getYear ()I
    ALOAD 0
    GETFIELD ru/butenko/springdatatest/Dog.birthdate : Ljava/time/LocalDate;
    INVOKEVIRTUAL java/time/LocalDate.getYear ()I
    ISUB
    IRETURN
    L1
    LOCALVARIABLE this Lru/butenko/springdatatest/Dog; L0 L1 0
    MAXSTACK = 2
    MAXLOCALS = 1
}
```

Типы данных, переменные, массивы, операции, управляющие операторы

- byte, short, int, long
- float, double
- boolean
- char
- Object
- int x[]
- арифметические операции
- операции отношения
- логические операции
- If, switch
- while, do while, for

Структура проекта



Class Syntax

Наследование:

```
public class Dog extends Animal {  
    // код класса  
}
```

Имплементация:

```
public class Cat implements Eatable {  
    // код класса  
}
```

Collections Framework

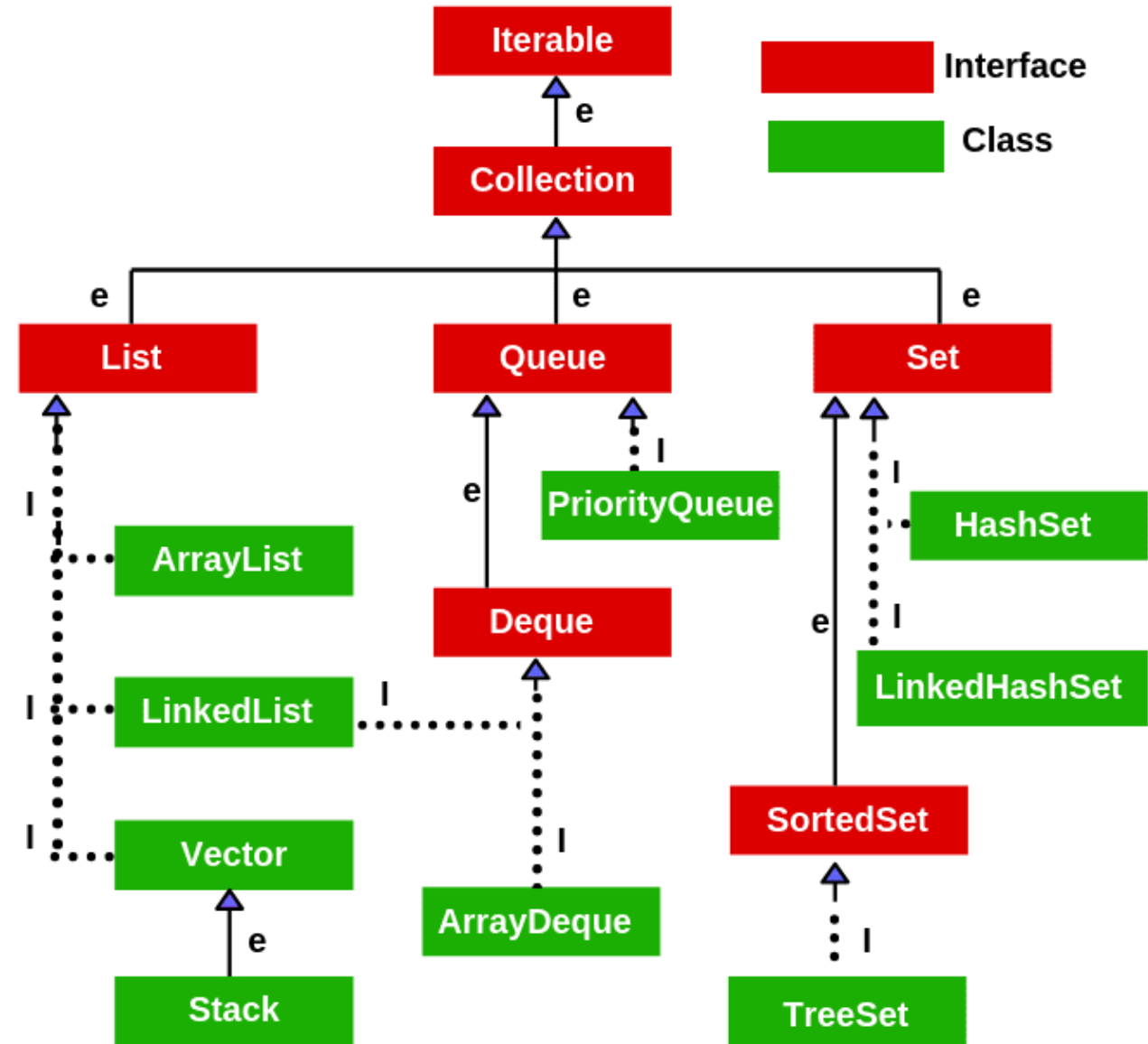
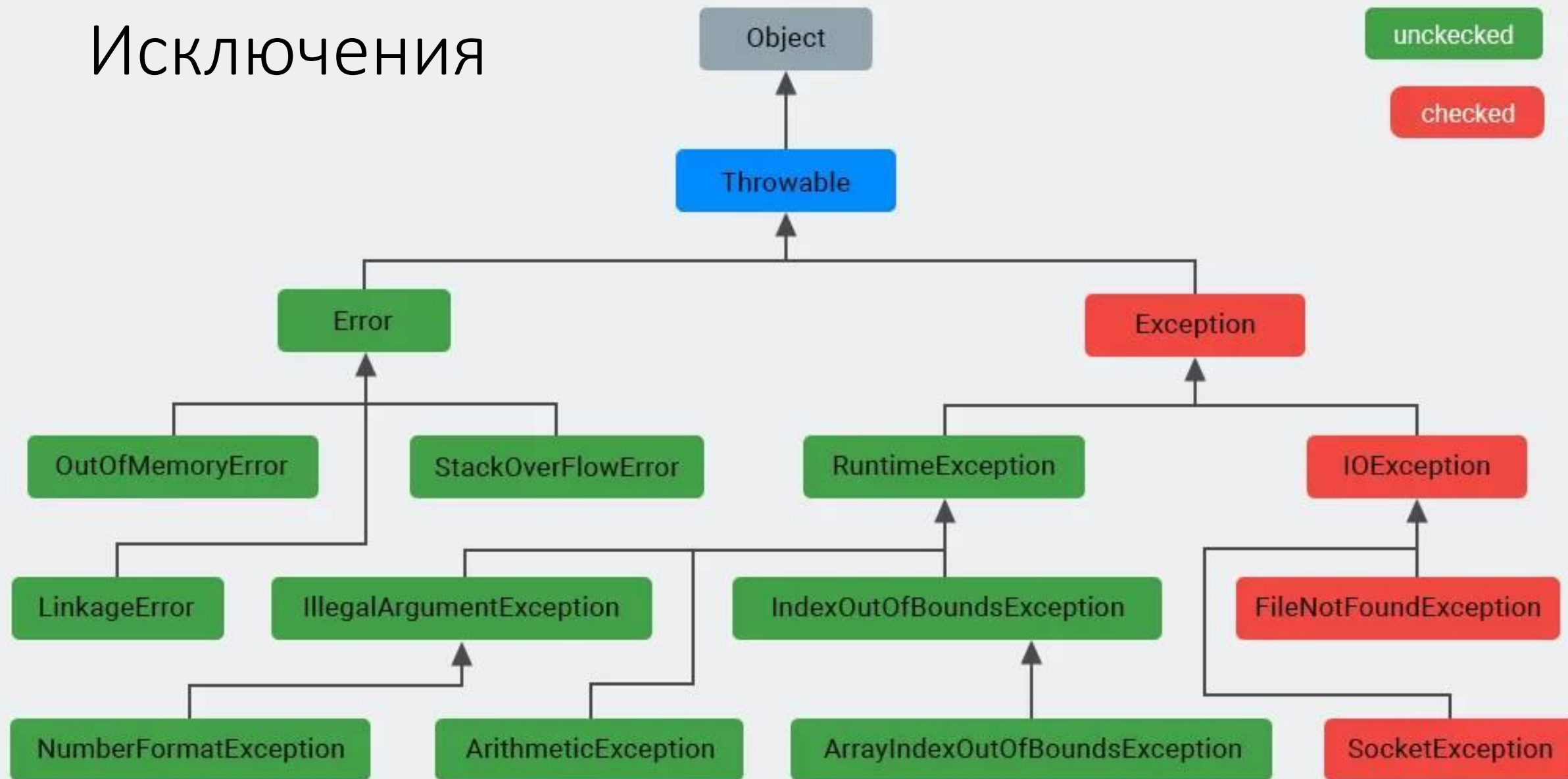


Fig: Collection Hierarchy in Java

Исключения



Исключения

Важно понимать, что проверка на checked исключения происходит в момент компиляции (compile-time checking)

перехват исключений (catch) происходит в момент выполнения (runtime checking)

```
public class App {  
    // пугаем тем что можем выбросить Exception  
    public static void main(String[] args) throws Exception {  
        throw new Exception();  
    }  
}
```

JavaDoc

<https://habr.com/ru/post/565342/>

```
import java.time.LocalDate;

/**
 * Класс, описывающий объект "Собака".
 */
public class Dog {
    public String name;
    private LocalDate birthdate;

    /**
     * Метод, который вычисляет возраст собаки на основе ее даты рождения.
     *
     * @return int возраст собаки.
     */
    public int calculateAge() {
        return LocalDate.now().getYear() - this.birthdate.getYear();
    }

    /**
     * Метод, который задает кличку собаки.
     *
     * @param name - кличка, которая будет дана собаке.
     */
    public void setName(String name) {
        this.name = name;
    }

    /**
     * Метод, который выбрасывает исключение.
     *
     * @throws Exception - базовое исключение, использованное для демонстрации тега
     */
    public void doException() throws Exception{
        throw new Exception("Exception!");
    }
}
```